

Tutorial de Git y GitHub con GitHub Desktop

Programación · Ingeniería Biomédica · Sesión práctica de 2 horas

Presentación

Al terminar esta sesión usted sabrá guardar versiones de su código, subirlas a GitHub y trabajar en equipo sobre un mismo proyecto, todo desde GitHub Desktop y sin usar la terminal.

Lo que va a aprender

- Qué es un repositorio y qué es un *commit*.
- Crear un repositorio, hacer cambios y revisar el historial.
- Publicar su trabajo en GitHub y mantenerlo sincronizado.
- Trabajar con ramas y *pull requests* junto a un compañero.

Requisitos previos

- Computador con Windows o macOS y conexión a internet.
- Un editor de código (por ejemplo, VS Code) y un compilador de C++ (g++ de MinGW-w64 o el que se use en el curso).
- Idealmente, cuenta de GitHub creada y GitHub Desktop instalado antes de la clase (ver Parte 2).

Parte 1: Conceptos básicos (15 min)

Git guarda fotografías de su proyecto a lo largo del tiempo; GitHub es el sitio web donde se guarda una copia de esas fotografías para compartirlas.

Piense en la carpeta con archivos como `proyecto_final_v3_AHORA_SI.cpp`: Git evita ese caos. En lugar de copiar archivos con nombres distintos, usted registra cada versión con una descripción y puede volver a cualquiera de ellas.

Vocabulario mínimo

Término	Qué significa	Analogía
Git	Programa que registra el historial de cambios de una carpeta	El cuaderno de laboratorio
Repositorio (repo)	Carpeta de proyecto cuyo historial controla Git	El proyecto completo con su bitácora
Commit	Fotografía guardada del proyecto, con un mensaje	Una entrada fechada y firmada en la bitácora
GitHub	Servicio web que aloja repositorios	Un archivo compartido en la nube

Término	Qué significa	Analogía
GitHub Desktop	Aplicación gráfica para usar Git y GitHub	Botones en lugar de comandos
Remoto (<i>origin</i>)	La copia del repo que vive en GitHub	La copia de respaldo del laboratorio
Push / Pull	Enviar commits a GitHub / traer commits desde GitHub	Subir / bajar la bitácora
Rama (<i>branch</i>)	Línea de trabajo paralela al proyecto principal	Un experimento aparte que luego se integra

El ciclo de trabajo que repetirá siempre

1. Edita archivos en su editor.
2. GitHub Desktop le muestra qué cambió.
3. Escribe un mensaje y hace *commit* (guarda la fotografía en su computador).
4. Hace *push* para enviar sus commits a GitHub.

Pregunta para la clase: ¿por qué es útil en un proyecto biomédico saber exactamente qué versión del código produjo un resultado?

Parte 2: Cuenta de GitHub y GitHub Desktop (15 min)

GitHub Desktop inicia sesión a través del navegador, así que no necesita crear llaves SSH ni tokens. Tampoco hace falta instalar Git por separado: GitHub Desktop trae su propia copia.

Crear la cuenta

1. Entre a github.com y pulse *Sign up*.
2. Use su correo institucional de la UIS y un nombre de usuario profesional (por ejemplo, [ana-rojas-bio](#)); lo verán futuros empleadores.
3. Confirme el correo desde el mensaje que le llega.
4. Opcional: solicite el paquete gratuito para estudiantes en education.github.com.

Instalar GitHub Desktop

1. Descárguelo desde desktop.github.com (Windows o macOS) e instálelo.
2. Al abrirlo por primera vez, pulse *Sign in to GitHub.com*; se abrirá el navegador para autorizar la aplicación.
3. Vuelva a GitHub Desktop y confirme su nombre y correo en la pantalla *Configure Git*.

Si la configuración se omitió, la encuentra en *File* → *Options* (Windows) o *GitHub Desktop* → *Settings* (macOS), secciones *Accounts* y *Git*.

Dato útil sobre el correo

Cada commit queda firmado con un nombre y un correo. Si prefiere no exponer su correo, en la sección *Git* elija la dirección que termina en `@users.noreply.github.com`.

Nota: en Linux no existe versión oficial de GitHub Desktop. Quien use Linux puede trabajar con un compañero o usar la versión comunitaria mantenida en GitHub por *shiftkey*.

Punto de control: todos ven su usuario en *File* → *Options* → *Accounts* antes de continuar.

Parte 3: Primer repositorio y primer commit (15 min)

Usted va a crear el repositorio `calculadora-imc` y guardar en él un pequeño programa en C++.

Crear el repositorio

1. En GitHub Desktop: *File* → *New repository* (**Ctrl+N**).
2. Complete los campos:
 - *Name*: `calculadora-imc`
 - *Description*: Cálculo del índice de masa corporal
 - *Local path*: una carpeta que recuerde, por ejemplo `Documentos/GitHub`
 - Marque *Initialize this repository with a README*
 - *Git ignore*: elija **C++**
3. Pulse *Create repository*. Git ya hizo un primer commit automático (*Initial commit*).
4. Pulse *Show in Explorer* (o *Show in Finder*) para ver la carpeta creada.

Agregar el primer archivo

Abra la carpeta en su editor y cree `imc.cpp` con este contenido:

```
#include <iostream>

double calcularImc(double pesoKg, double estaturaM) {
    return pesoKg / (estaturaM * estaturaM);
}

int main() {
    double peso, estatura;
    std::cout << "Peso (kg): ";
    std::cin >> peso;
    std::cout << "Estatura (m): ";
    std::cin >> estatura;
    std::cout << "IMC: " << calcularImc(peso, estatura) << std::endl;
    return 0;
}
```

Compílelo y pruébelo: `g++ imc.cpp -o imc`

Hacer el commit

1. Vuelva a GitHub Desktop. En la pestaña *Changes* aparece `imc.cpp` (archivo nuevo). El ejecutable `imc.exe` no aparece: la plantilla `.gitignore` de C++ lo excluye, porque en un repositorio se guarda el código fuente, no los binarios.
2. Abajo a la izquierda, en *Summary*, escriba: **Agrega cálculo básico del IMC.**
3. Pulse *Commit to main*.
4. Abra la pestaña *History*: ahora hay dos commits.

Buenos mensajes de commit

- Empiezan con un verbo y dicen qué hace el cambio: **Corrige división por cero en IMC.**
- Evite mensajes como `cambios`, `asdf` o `final final`.

Punto de control: cada estudiante ve dos commits en *History*.

Parte 4: Cambios, historial y deshacer (20 min)

GitHub Desktop muestra línea por línea qué cambió (el *diff*) y permite volver atrás sin perder trabajo.

Modificar y comparar

Agregue `#include <string>` al inicio de `imc.cpp` y esta función antes de `main`:

```
std::string clasificarImc(double imc) {  
    if (imc < 18.5) {  
        return "Bajo peso";  
    } else if (imc < 25.0) {  
        return "Normal";  
    } else if (imc < 30.0) {  
        return "Sobrepeso";  
    }  
    return "Obesidad";  
}
```

Cambie `main` para imprimir también la clasificación:

```
double imc = calcularImc(peso, estatura);  
std::cout << "IMC: " << imc  
          << " (" << clasificarImc(imc) << ")" << std::endl;
```

Compile, pruebe y observe la pestaña *Changes*:

- Las líneas en verde (+) son nuevas; las rojas (-) fueron eliminadas.
- La casilla junto a cada archivo decide si entra o no en el próximo commit.

Haga commit con el mensaje **Agrega clasificación del IMC según la OMS.**

Commits pequeños y frecuentes

Regla práctica: un commit por cada idea terminada. Agregue ahora un archivo `datos_pacientes.csv` con tres filas de datos inventados y haga un commit separado para él.

Explorar el historial

1. Abra *History* y haga clic en cada commit para ver sus cambios.
2. Observe autor, fecha y mensaje: esa es la trazabilidad del proyecto.

Tres formas de deshacer

Situación	Qué hacer en GitHub Desktop
Cambié un archivo pero no he hecho commit y quiero descartarlo	Clic derecho sobre el archivo en <i>Changes</i> → <i>Discard changes</i>
Acabo de hacer un commit y me equivoqué en el mensaje o faltó algo	Botón <i>Undo</i> que aparece abajo a la izquierda justo después del commit
Un commit anterior introdujo un error	Clic derecho sobre el commit en <i>History</i> → <i>Revert changes in commit</i> (crea un commit nuevo que lo anula)

Mini-reto (5 min): introduzca a propósito un error de compilación en `imc.cpp` (por ejemplo, borre un punto y coma), haga commit y luego anúlelo con *Revert*. Revise el historial: los dos commits quedan registrados.

Parte 5: Publicar, push, pull y clonar (20 min)

Hasta ahora todo vive solo en su computador; publicar crea la copia en GitHub, y *push* y *pull* la mantienen sincronizada.

Publicar el repositorio

1. Pulse el botón azul *Publish repository* en la barra superior.
2. Deje marcada *Keep this code private* si no quiere que sea público.
3. Pulse *Publish repository* y luego *Repository* → *View on GitHub*: ahí están su código, su README y su historial.

Importante en biomédica: nunca suba datos reales de pacientes, ni siquiera a un repositorio privado. Use datos sintéticos o anonimizados.

Enviar cambios (push)

1. Edite `README.md`: agregue una línea que explique cómo compilar y ejecutar `imc.cpp`.
2. Haga commit.
3. El botón superior cambia a *Push origin* con un número: son los commits que GitHub aún no tiene. Púselo.

4. Recargue la página del repositorio en el navegador y compruebe el cambio.

Traer cambios (pull)

1. En la página del repo en GitHub, abra **README.md**, pulse el lápiz (*Edit*) y agregue su nombre como autor. Guarde con *Commit changes*.
2. En GitHub Desktop pulse *Fetch origin*: Desktop pregunta a GitHub si hay novedades.
3. El botón cambia a *Pull origin*. Púlselo y abra el README en su editor: el cambio ya está.

Hábito clave: antes de empezar a trabajar, *Fetch/Pull*; al terminar, *Commit y Push*.

Clonar un repositorio existente

Clonar es descargar un repo de GitHub con todo su historial.

1. *File* → *Clone repository*.
2. En la pestaña *URL*, pegue la dirección del repositorio que el profesor comparta en el tablero.
3. Elija la carpeta local y pulse *Clone*.

Punto de control: cada estudiante comparte el enlace a su repo **calculadora-imc** (o lo muestra en pantalla si es privado).

Parte 6: Ramas y pull requests (15 min)

Una rama le permite probar una idea sin tocar la versión que funciona (**main**); un *pull request* es la propuesta formal para integrarla.

Crear una rama y trabajar en ella

1. *Branch* → *New branch* (**Ctrl+Shift+N**) y nómbrela **conversion-libras**.
2. Confirme arriba que *Current branch* dice **conversion-libras**.
3. Agregue a **imc.cpp** una función que convierta libras a kilogramos (1 lb = 0,4536 kg):

```
double librasAKilogramos(double libras) {  
    return libras * 0.4536;  
}
```

1. Compile, haga commit y pulse *Publish branch*.

Pruebe esto: cambie a **main** con el menú *Current branch* y mire **imc.cpp** en su editor. La función nueva desaparece; vuelva a la rama y reaparece.

Abrir y aceptar un pull request

1. Estando en la rama, *Branch* → *Create pull request*; se abre el navegador.

2. Escriba un título y una descripción breve de qué agrega y cómo probarlo. Pulse *Create pull request*.
3. Revise la pestaña *Files changed*: así lo verá quien revise su código.
4. Pulse *Merge pull request* y luego *Confirm merge*.
5. En GitHub Desktop cambie a **main** y pulse *Fetch origin* → *Pull origin*. La función ya está en **main**.
6. Opcional: borre la rama ya integrada con *Branch* → *Delete*.

¿Por qué tantos pasos para un cambio tan pequeño? Porque en equipo el pull request es el lugar donde otra persona revisa, comenta y aprueba antes de que el cambio llegue a la versión principal.

Parte 7: Ejercicio final en parejas (15 min)

En parejas, construyan un pequeño módulo de signos vitales donde cada persona aporta una función mediante su propia rama y pull request.

Roles: la persona A es dueña del repositorio; la persona B es colaboradora.

1. **A** crea y publica el repositorio **signos-vitales** (plantilla **.gitignore** de C++) con README y un archivo **signos.cpp** que solo contiene **main**.
2. **A** entra al repo en GitHub: *Settings* → *Collaborators* → *Add people* e invita a **B** por su usuario.
3. **B** acepta la invitación (llega por correo o en github.com/notifications) y clona el repo con *File* → *Clone repository*.
4. Cada uno crea su rama desde **main**:
 - **A**: rama **temperatura**, función **double fahrenheitACelsius(double f)** que aplica $(F - 32) \times 5/9$.
 - **B**: rama **frecuencia-cardiaca**, función **std::string clasificarFrecuencia(int lpm)**: menos de 60 lpm, bradicardia; de 60 a 100, normal; más de 100, taquicardia.
5. Cada uno llama su función desde **main**, compila, hace commit, publica su rama y abre un pull request.
6. Cada uno revisa el pull request del otro, deja al menos un comentario y lo aprueba.
7. Integren ambos pull requests con *Merge*.
8. Ambos hacen *Pull origin* en **main** y verifican que **signos.cpp** compila y tiene las dos funciones.

Si aparece un conflicto: ocurre cuando ambos modificaron las mismas líneas (muy probable aquí, porque los dos editan **main**). GitHub mostrará *This branch has conflicts*; pulsen *Resolve conflicts*, dejen el código correcto entre las marcas **<<<<<<** y **>>>>>>**, borren las marcas y confirmen. Si pasa, llamen al profesor: es la mejor oportunidad para aprenderlo.

Entrega: el enlace al repo **signos-vitales** con los dos pull requests cerrados.

Problemas frecuentes y buenas prácticas

Casi todos los tropiezos de la primera sesión se resuelven con *Fetch origin* antes de trabajar y leyendo el mensaje que muestra GitHub Desktop.

Síntoma	Causa probable	Solución
No aparece <i>Push origin</i> , solo <i>Publish repository</i>	El repo aún no existe en GitHub	Publicarlo (Parte 5)
<i>Push</i> rechazado: hay cambios en GitHub que usted no tiene	Alguien más (o usted desde la web) hizo commits	<i>Fetch origin</i> → <i>Pull origin</i> y luego <i>Push</i>
El archivo no aparece en <i>Changes</i>	Se guardó en otra carpeta o está en <i>.gitignore</i>	<i>Repository</i> → <i>Show in Explorer</i> y verificar la ruta
Aparecen ejecutables o archivos <i>.o</i> en <i>Changes</i>	El repo se creó sin la plantilla <i>.gitignore</i> de C++	Clic derecho sobre el archivo → <i>Ignore file</i>
Los commits no aparecen asociados a su cuenta en GitHub	El correo del commit no coincide con el de la cuenta	Corregir el correo en <i>Options</i> → <i>Git</i>
No aparece el repo del compañero al clonar	Aún no aceptó la invitación de colaborador	Aceptarla en github.com/notifications
Hice commit en <i>main</i> en vez de en mi rama	Olvidé cambiar de rama	Si no hice push: <i>Undo</i> , cambiar de rama y volver a hacer commit

Buenas prácticas

- Un commit por idea terminada, con mensaje que empiece por verbo.
- *Pull* al empezar, *Push* al terminar.
- Solo código fuente en el repositorio: nada de ejecutables ni archivos objeto.
- Nunca datos reales de pacientes, contraseñas ni claves de API en un repositorio.
- Un README que diga qué hace el proyecto y cómo compilarlo y ejecutarlo.
- Trabajo nuevo en una rama; a *main* solo llega lo que compila y funciona.

Para seguir aprendiendo

- Documentación de GitHub Desktop en español: docs.github.com/es/desktop.
- GitHub Skills, cursos interactivos gratuitos: skills.github.com.
- Más adelante: los mismos conceptos desde la terminal (*git add*, *git commit*, *git push*) y autenticación con llaves SSH.